

CENTRO UNIVERSITÁRIO DE BRASÍLIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB

BRUNO DOS SANTOS
CARLOS EMANUEL
GUSTAVO AUGUSTO

MODELO DE DADOS

CENTRALBEER
SISTEMA WEB PARA GESTÃO DE ESTOQUE, VENDAS E CAIXA
PARA DISTRIBUIDORAS DE BEBIDAS

BRASÍLIA - DF
2026

MODELO DE DADOS
CENTRALBEER
SISTEMA WEB PARA GESTÃO DE ESTOQUE, VENDAS E CAIXA
PARA DISTRIBUIDORAS DE BEBIDAS

Modelo de Dados apresentado à disciplina Desenvolvimento Web, do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário de Brasília (CEUB), como parte da documentação da Fase 1 do projeto CentralBeer.

Professor: Felipe Pires Ferreira

BRASÍLIA - DF
2026

SUMÁRIO

CONTROLE DO DOCUMENTO	4
RESUMO EXECUTIVO	4
1 OBJETIVO E ESCOPO DA MODELAGEM.....	4
2 PREMISSAS DE MODELAGEM E TIPOS.....	5
3 DIAGRAMA ENTIDADE-RELACIONAMENTO	6
4 MODELO RELACIONAL DETALHADO	7
5 DICIONÁRIO DE DADOS	8
6 CARDINALIDADES E RELACIONAMENTOS	14
7 RESTRIÇÕES DE INTEGRIDADE E REGRAS	15
8 ÍNDICES E ESTRATÉGIA DE PERSISTÊNCIA	16
9 FLUXOS DE DADOS PERSISTENTES.....	17
10 SITUAÇÃO, LIMITAÇÕES E REFERÊNCIA	18

Controle do documento

Campo	Registro
Versão	1.0
Data de referência	03/10/2026
Status documental	Fase 1 - modelo lógico proposto; migrations e tabelas ainda não implementadas.
Escopo	Modelo de dados do CentralBeer com entidades/tabelas, atributos, tipos, chaves, cardinalidades, restrições e índices relevantes.
Base documental	README.md do repositório público Bruno-Kowalski/Central-Beer. Os PDFs herdados do template de banco de dados não são utilizados como fonte do domínio do CentralBeer.
Ferramenta do diagrama	Graphviz, com representação entidade-relacionamento e esquema relacional.
Banco planejado	SQLite acessado pelo Django ORM.

Resumo executivo

O modelo de dados do CentralBeer foi estruturado para sustentar cadastro e busca de produtos, estoque em unidades, vendas por unidade ou caixa, pagamentos em até duas formas, abertura e fechamento de caixa, retiradas, reforços, cancelamentos e rastreabilidade das movimentações. O desenho preserva o histórico de vendas e movimentações e mantém os valores monetários em centavos inteiros.

A modelagem adota dez entidades principais: Usuário, Categoria, Produto, Caixa, Movimentação de Caixa, Conferência de Caixa, Venda, Item de Venda, Pagamento e Movimentação de Estoque. Tabelas internas do próprio Django para sessão, permissões e tokens podem ser adicionadas pelo framework na Fase 2 e não fazem parte do núcleo de domínio representado neste documento.

1. Objetivo e escopo da modelagem

O objetivo é definir uma base relacional coerente com as funcionalidades e decisões técnicas já registradas para o CentralBeer. Como o projeto está em fase documental, o modelo descreve a estrutura planejada para implementação com Django e SQLite; nomes finais de classes, campos e migrations poderão ser refinados na Fase 2 sem alterar as regras de domínio aqui documentadas.

2. Premissas de modelagem e tipos

O README do CentralBeer define SQLite como banco da primeira versão, uso do Django ORM, valores monetários em centavos inteiros, quantidades inteiras, um único caixa aberto por vez, preservação de histórico de venda e transações atômicas. O modelo lógico abaixo transforma essas decisões em tabelas e restrições verificáveis.

Premissa	Aplicação no modelo
Valores monetários	Campos monetários usam INTEGER em centavos, evitando ponto flutuante.
Estoque	Saldo, entradas, saídas e conversões são representados em unidades inteiras.
Venda por caixa	ITEM_VENDA guarda o fator de conversão utilizado no momento da venda e a quantidade efetiva baixada.
Histórico de preço	O preço unitário é copiado para ITEM_VENDA para não mudar o passado quando PRODUTO for atualizado.
Produtos com histórico	São inativados; relações históricas não dependem da exclusão física.
Caixa único	Regra de no máximo um registro CAIXA com status ABERTO.
Pagamento dividido	Cada venda possui uma ou duas formas; a mesma forma não pode aparecer duas vezes.
Integridade transacional	Venda, itens, pagamentos e movimentações de estoque devem ser persistidos na mesma transação.
Rastreabilidade	Movimentações guardam usuário, data/hora e motivo quando aplicável.
Django Auth	USUARIO representa a identidade do domínio; a implementação poderá usar auth_user ou modelo customizado do Django.

2.1 Convenções de tipos

Tipo lógico	Uso
INTEGER	Chaves primárias, chaves estrangeiras, centavos e quantidades.
VARCHAR(n)	Códigos, nomes, status, perfis e formas de pagamento.
TEXT	Motivos e justificativas de tamanho variável.
BOOLEAN	Indicadores de ativo/inativo.
DATETIME	Data e hora de eventos de negócio.

3. Diagrama Entidade-Relacionamento

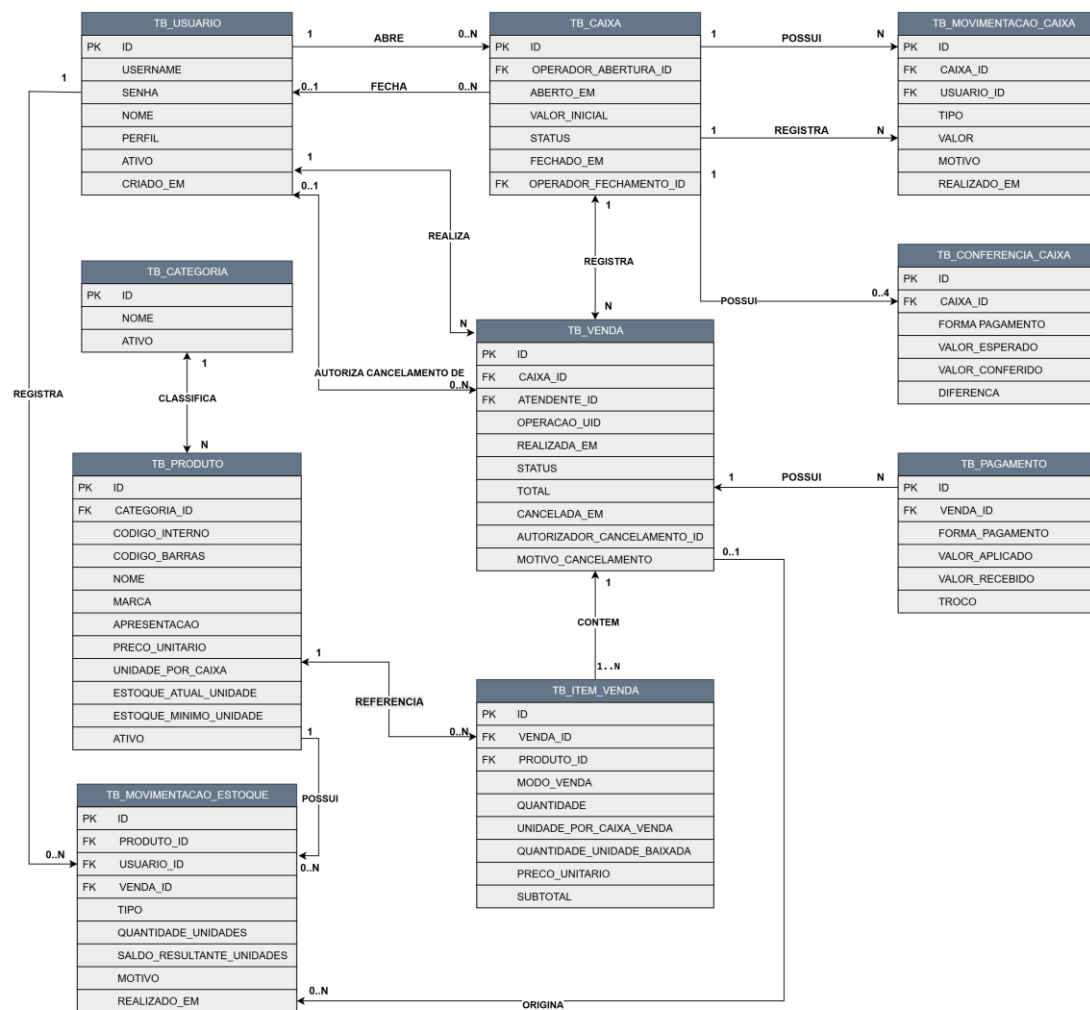


Figura 1 - Diagrama Entidade-Relacionamento do CentralBeer, com cardinalidades. Produzido em draw.io. Fonte: elaboração própria com base na documentação do projeto (2026).

4. Modelo relacional detalhado

A visão abaixo resume as relações físicas planejadas. O dicionário de dados da seção 5 apresenta todos os atributos, tipos e restrições de cada tabela.

Tabela	Chave primária	Chaves estrangeiras	Restrições principais
USUARIO	id	-	username único; perfil ADMINISTRADOR/ATENDENTE; ativo.
CATEGORIA	id	-	nome único; ativo.
PRODUTO	id	categoria_id	codigo_interno único; codigo_barras único quando informado; saldos ≥ 0 ; unidades_por_caixa > 0 .
CAIXA	id	operador_abertura_id; operador_fechamento_id	status ABERTO/FECHADO; no máximo um caixa ABERTO.
MOVIMENTACAO_CAIXA	id	caixa_id; usuario_id	tipo RETIRADA/REFORCO; valor > 0 ; motivo obrigatório.
CONFERENCIA_CAIXA	id	caixa_id	UNIQUE(caixa_id, forma_pagamento).
VENDA	id	caixa_id; atendente_id; autorizador_cancelamento_id	operacao_uid único; total > 0 ; status FINALIZADA/CANCELADA.
ITEM_VENDA	id	venda_id; produto_id	quantidades > 0 ; preço e fator de conversão preservados como snapshot.
PAGAMENTO	id	venda_id	UNIQUE(venda_id, forma); 1 a 2 formas por venda.
MOVIMENTACAO_ESTOQUE	id	produto_id; usuario_id; venda_id	quantidade > 0 ; saldo resultante ≥ 0 ; tipo controlado.

4.1 Normalização e histórico

O modelo separa itens de venda, pagamentos, movimentações de estoque e conferências de caixa para evitar repetição de grupos de dados e preservar o histórico. Informações que precisam permanecer iguais ao momento da venda, como preço e fator de conversão, são copiadas para ITEM_VENDA de forma intencional.

5. Dicionário de dados

As subseções a seguir detalham os atributos do modelo lógico. PK indica chave primária, FK chave estrangeira e UQ restrição de unicidade. Campos descritos como opcionais podem receber NULL enquanto a condição de negócio correspondente não ocorrer.

5.1 USUARIO

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
UQ	username	VARCHAR(150)	Identificador de login; único e obrigatório.
-	senha_hash	VARCHAR(128)	Senha armazenada somente como hash do Django.
-	nome	VARCHAR(150)	Nome de exibição do usuário.
-	perfil	VARCHAR(20)	ADMINISTRADOR ou ATENDENTE.
-	ativo	BOOLEAN	Indica se a conta pode ser utilizada.
-	criado_em	DATETIME	Data/hora de criação.

Observação de implementação

A tabela de usuário é uma abstração do domínio. Em Django, a implementação poderá reutilizar o modelo de autenticação do framework e complementar o perfil necessário ao CentralBeer, sem armazenar senhas em texto puro.

5.2 CATEGORIA

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
UQ	nome	VARCHAR(80)	Nome da categoria; único.
-	ativo	BOOLEAN	Permite inativação sem apagar histórico.

5.3 PRODUTO

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	categoria_id	INTEGER	Referência CATEGORIA(id).
UQ	codigo_interno	VARCHAR(40)	Código interno único.
UQ	codigo_barras	VARCHAR(32)	Código de barras; opcional e único quando informado.
-	nome	VARCHAR(150)	Nome do produto.
-	marca	VARCHAR(100)	Marca; opcional, inclusive quando sugerida pela integração.
-	apresentacao	VARCHAR(100)	Apresentação/embalagem; opcional.
-	preco_unitario_centavos	INTEGER	Preço unitário em centavos; ≥ 0 .
-	unidades_por_caixa	INTEGER	Fator de conversão caixa -> unidades; > 0 .
-	estoque_atual_unidades	INTEGER	Saldo atual em unidades; ≥ 0 .
-	estoque_minimo_unidades	INTEGER	Limite para alerta de estoque; ≥ 0 .
-	ativo	BOOLEAN	Produto com histórico é inativado, não apagado.

Regras específicas de produto

- codigo_interno é obrigatório e único.
- codigo_barras é opcional; quando informado, não deve se repetir.

- unidades_por_caixa deve ser maior que zero.
- estoque_atual_unidades e estoque_minimo_unidades não podem ser negativos.
- Produtos já utilizados em histórico devem ser inativados em vez de excluídos.

5.4 CAIXA

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	operador_abertura_id	INTEGER	Referência USUARIO(id).
-	aberto_em	DATETIME	Data/hora de abertura.
-	valor_inicial_centavos	INTEGER	Valor inicial para troco; ≥ 0 .
-	status	VARCHAR(10)	ABERTO ou FECHADO.
FK	operador_fechamento_id	INTEGER	Referência USUARIO(id); nulo enquanto aberto.
-	fechado_em	DATETIME	Data/hora de fechamento; nulo enquanto aberto.

5.5 MOVIMENTACAO_CAIXA

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	caixa_id	INTEGER	Referência CAIXA(id).
FK	usuario_id	INTEGER	Usuário responsável.
-	tipo	VARCHAR(10)	RETIRADA ou REFORCO.
-	valor_centavos	INTEGER	Valor movimentado; > 0 .
-	motivo	TEXT	Justificativa obrigatória.
-	realizado_em	DATETIME	Data/hora da operação.

Regra de caixa aberto

O modelo deve impedir mais de um caixa aberto simultaneamente. Em SQLite, a regra pode ser materializada com índice único parcial sobre o status ABERTO; adicionalmente, o serviço de aplicação deve validar a condição dentro da transação de abertura.

5.6 CONFERENCIA_CAIXA

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	caixa_id	INTEGER	Referência CAIXA(id).
-	forma_pagamento	VARCHAR(10)	DINHEIRO, PIX, DEBITO ou CREDITO.
-	valor_esperado_cen tavos	INTEGER	Total calculado pelo sistema; >= 0.
-	valor_conferido_cen tavos	INTEGER	Valor informado no fechamento; >= 0.
-	diferenca_centavos	INTEGER	Conferido - esperado; pode ser negativo.
UQ	(caixa_id, forma_pagamento)	UNIQUE	Uma conferência por forma em cada caixa.

5.7 VENDA

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	caixa_id	INTEGER	Referência CAIXA(id).
FK	atendente_id	INTEGER	Referência USUARIO(id).
UQ	operacao_uid	VARCHAR(64)	Identificador único contra reenvio/duplicidade.
-	realizada_em	DATETIME	Data/hora da finalização.
-	status	VARCHAR(12)	FINALIZADA ou CANCELADA.
-	total_centavos	INTEGER	Total da venda; > 0.
-	cancelada_em	DATETIME	Preenchido quando houver cancelamento.
FK	autorizador_cancela mento_id	INTEGER	Administrador que autorizou; opcional.
-	motivo_cancelamen to	TEXT	Motivo do cancelamento; opcional enquanto ativa.

Cancelamento

Campos de cancelamento permanecem nulos em vendas ativas. Quando status for CANCELADA, a aplicação registra data/hora e o administrador autorizador, preservando a venda original e gerando os estornos necessários.

5.8 ITEM_VENDA

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	venda_id	INTEGER	Referência VENDA(id).
FK	produto_id	INTEGER	Referência PRODUTO(id).
-	modo_venda	VARCHAR(10)	UNIDADE ou CAIXA.
-	quantidade	INTEGER	Quantidade de unidades comerciais informadas; > 0.
-	unidades_por_caixa_snapshot	INTEGER	Fator copiado do produto no momento da venda; > 0.
-	quantidade_unidades_baixas	INTEGER	Quantidade efetiva descontada do estoque; > 0.
-	preco_unitario_centavos_snapshot	INTEGER	Preço unitário copiado para preservar histórico; >= 0.
-	subtotal_centavos	INTEGER	Subtotal do item; >= 0.

5.9 PAGAMENTO

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	venda_id	INTEGER	Referência VENDA(id).
-	forma	VARCHAR(10)	DINHEIRO, PIX, DEBITO ou CREDITO.
-	valor_aplicado_centavos	INTEGER	Parcela do total coberta pela forma; > 0.
-	valor_recebido_centavos	INTEGER	Valor entregue em dinheiro; nulo nas demais formas.
-	troco_centavos	INTEGER	Troco calculado somente para DINHEIRO; >= 0.
UQ	(venda_id, forma)	UNIQUE	Impede repetir a mesma forma na mesma venda.

Pagamento dividido

A restrição UNIQUE(venda_id, forma) evita repetir a mesma forma. A regra de no máximo duas formas por venda e a igualdade entre a soma de valor_aplicado_centavos e o total da venda devem ser validadas na camada de serviço dentro da mesma transação de finalização.

5.10 MOVIMENTACAO_ESTOQUE

Chave	Atributo	Tipo lógico	Descrição / restrição
PK	id	INTEGER	Chave primária.
FK	produto_id	INTEGER	Referência PRODUTO(id).
FK	usuario_id	INTEGER	Usuário responsável pela operação.
FK	venda_id	INTEGER	Referência VENDA(id); nula em entrada/ajuste manual.
-	tipo	VARCHAR(18)	ENTRADA, VENDA, AJUSTE_POSITIVO, AJUSTE_NEGATIVO ou ESTORNO.
-	quantidade_unidades	INTEGER	Quantidade movimentada; > 0.
-	saldo_resultante_unidades	INTEGER	Saldo após a movimentação; >= 0.
-	motivo	TEXT	Obrigatório em ajustes; opcional nos fluxos automáticos.
-	realizado_em	DATETIME	Data/hora da movimentação.

Rastreabilidade do estoque

Entradas manuais, ajustes, baixas de venda e estornos usam o mesmo histórico de movimentações. A associação opcional com VENDA identifica baixas e estornos originados do ponto de venda, enquanto usuario_id e realizado_em permitem identificar responsável e momento da operação.

5.11 Tabelas técnicas fora do núcleo do diagrama

O Django poderá criar tabelas técnicas para migrations, sessões, permissões, grupos e tokens da API. Elas são responsabilidades do framework e não alteram as entidades centrais do domínio apresentadas neste documento.

6. Cardinalidades e relacionamentos

Relacionamento	Cardinalidade	Interpretação
CATEGORIA - PRODUTO	1 : 0..N	Uma categoria pode classificar vários produtos; cada produto pertence a uma categoria.
PRODUTO - MOVIMENTACAO_ESTOQUE	1 : 0..N	Um produto possui histórico de várias movimentações.
USUARIO - MOVIMENTACAO_ESTOQUE	1 : 0..N	Cada movimentação identifica um usuário responsável.
VENDA - MOVIMENTACAO_ESTOQUE	0..1 : 0..N	Movimentações manuais não precisam de venda; uma venda pode gerar várias baixas/estornos.
USUARIO - CAIXA	1 : 0..N	Usuários podem abrir/fechar caixas ao longo do tempo.
CAIXA - MOVIMENTACAO_CAIXA	1 : 0..N	Um caixa pode ter várias retiradas e reforços.
CAIXA - CONFERENCIA_CAIXA	1 : 0..4	No fechamento pode existir uma conferência para cada uma das quatro formas de pagamento.
CAIXA - VENDA	1 : 0..N	Todas as vendas são vinculadas ao caixa em que ocorreram.
USUARIO - VENDA	1 : 0..N	Cada venda registra o atendente; cancelamentos podem registrar outro usuário autorizador.
VENDA - ITEM_VENDA	1 : 1..N	Uma venda finalizada possui um ou mais itens.
PRODUTO - ITEM_VENDA	1 : 0..N	Um produto pode aparecer em vários itens de vendas históricas.
VENDA - PAGAMENTO	1 : 1..2	Uma venda finalizada possui uma ou duas formas de pagamento.

6.1 Regras de exclusão referencial

Entidades históricas devem preferir comportamento restritivo (PROTECT/RESTRICT) para impedir que usuários, produtos, vendas ou caixas referenciados sejam apagados. Itens e pagamentos pertencem ao agregado de venda, mas como a própria venda deve ser preservada, a aplicação não deve oferecer exclusão física de vendas finalizadas.

7. Restrições de integridade e regras

Regra	Aplicação / validação
RI-01 - Dinheiro em centavos	Todos os valores monetários são INTEGER; nenhum cálculo usa ponto flutuante.
RI-02 - Quantidades positivas	Entradas, vendas e movimentações exigem quantidades inteiras maiores que zero.
RI-03 - Saldo não negativo	estoque_atual_unidades e saldo_resultante_unidades devem ser ≥ 0 ; finalização bloqueia estoque insuficiente.
RI-04 - Conversão válida	unidades_por_caixa > 0 ; venda por caixa calcula unidades baixadas pela quantidade x snapshot do fator.
RI-05 - Histórico imutável	ITEM_VENDA guarda preço e fator de conversão da operação.
RI-06 - Caixa único aberto	No máximo um CAIXA com status ABERTO.
RI-07 - Caixa de origem	Venda e cancelamento permanecem vinculados ao caixa original; cancelamento só enquanto ele estiver aberto.
RI-08 - Pagamentos	Venda possui 1 ou 2 registros; formas não se repetem; soma aplicada corresponde ao total.
RI-09 - Troco	Troco só é calculado em DINHEIRO e com base no valor recebido em espécie.
RI-10 - Idempotência	operacao_uid é único para impedir duplicação de venda em reenvio.
RI-11 - Ajustes justificados	Movimentações administrativas de estoque e caixa exigem motivo.
RI-12 - Cancelamento autorizado	Cancelamento integral exige administrador válido e preserva a venda original.
RI-13 - Atomicidade	Venda, itens, pagamentos, baixa/estorno de estoque e efeitos financeiros são persistidos na mesma transação.
RI-14 - Sem edição retroativa	Fechamentos concluídos permanecem preservados; diferenças ficam registradas em CONFERENCIA_CAIXA.

7.1 Restrições CHECK sugeridas

- perfil IN (ADMINISTRADOR, ATENDENTE).
- status de CAIXA IN (ABERTO, FECHADO).
- status de VENDA IN (FINALIZADA, CANCELADA).
- formas de pagamento IN (DINHEIRO, PIX, DEBITO, CREDITO).

- tipo de MOVIMENTACAO_CAIXA IN (RETIRADA, REFORCO).
- tipo de MOVIMENTACAO_ESTOQUE IN (ENTRADA, VENDA, AJUSTE_POSITIVO, AJUSTE_NEGATIVO, ESTORNO).

8. Índices e estratégia de persistência

Além das chaves primárias e índices normalmente criados para chaves estrangeiras, o modelo recomenda índices alinhados às pesquisas e relatórios previstos no CentralBeer.

Índice / restrição	Campos	Finalidade
UQ produto código interno	PRODUTO(codigo_interno)	Busca direta e prevenção de duplicidade.
UQ produto código de barras	PRODUTO(codigo_barras)	Leitura/busca de código de barras e prevenção de duplicidade quando preenchido.
IDX produto nome	PRODUTO(nome)	Busca de produtos por nome.
IDX produto categoria	PRODUTO(categoria_id, ativo)	Listagens e filtros por categoria.
IDX movimento estoque	MOVIMENTACAO_ESTOQUE(produto_id, realizado_em)	Histórico por produto e período.
IDX venda período	VENDA(realizada_em, status)	Histórico e relatórios de vendas.
IDX venda caixa	VENDA(caixa_id, realizada_em)	Resumo da sessão de caixa.
UQ venda operação	VENDA(operacao_uid)	Idempotência da finalização.
UQ pagamento forma	PAGAMENTO(venda_id, forma)	Uma ocorrência de cada forma por venda.
UQ conferência forma	CONFERENCIA_CAIXA(caixa_id, forma_pagamento)	Uma conferência por forma no fechamento.
UQ parcial caixa aberto	CAIXA(status) WHERE status = ABERTO	Impedir dois caixas abertos simultaneamente no SQLite.

8.1 Atualização do saldo

PRODUTO.estoque_atual_unidades mantém o saldo corrente para consulta rápida, enquanto MOVIMENTACAO_ESTOQUE mantém o histórico. Ambos devem ser atualizados na mesma transação. A coluna saldo_resultante_unidades auxilia conferência e auditoria de cada movimento.

8.2 Backup

O arquivo SQLite e seus backups não devem ser versionados. O planejamento prevê cópia consistente antes de migrations e implantações e após o fechamento diário.

9. Fluxos de dados persistentes

9.1 Entrada de mercadoria

1. Usuário seleciona PRODUTO e informa quantidade por unidade ou caixa.
2. A aplicação converte caixas para unidades usando PRODUTO.unidades_por_caixa.
3. Na mesma transação, atualiza PRODUTO.estoque_atual_unidades e cria MOVIMENTACAO_ESTOQUE do tipo ENTRADA.
4. O movimento registra usuário, quantidade, saldo resultante e data/hora.

9.2 Finalização de venda

1. A aplicação valida CAIXA aberto, produtos, saldo e formas de pagamento.
2. Cria VENDA com operacao_uid único e total em centavos.
3. Cria ITEM_VENDA para cada produto, copiando preço e fator de conversão utilizados.
4. Cria um ou dois PAGAMENTO, validando soma e eventual troco em dinheiro.
5. Atualiza saldos de PRODUTO e cria MOVIMENTACAO_ESTOQUE do tipo VENDA.
6. Toda a operação é confirmada em uma única transação; falha em qualquer etapa reverte o conjunto.

9.3 Cancelamento

1. Valida caixa de origem ainda ABERTO e credencial do administrador.
2. Atualiza VENDA para CANCELADA sem apagar o registro original.
3. Registra autorizador e data/hora do cancelamento.
4. Recompõe estoque por MOVIMENTACAO_ESTOQUE do tipo ESTORNO e ajusta totais usados no fechamento.

9.4 Fechamento de caixa

1. Consolida PAGAMENTO das vendas válidas e MOVIMENTACAO_CAIXA.
2. Grava CONFERENCIA_CAIXA por forma com valor esperado, conferido e diferença.
3. Atualiza CAIXA com operador de fechamento, data/hora e status FECHADO.

10. Situação, limitações e referência

10.1 Situação documental

Na data de referência, o repositório está na Fase 1 e o README informa que aplicação, testes e workflows serão implementados na Fase 2. Por isso, este documento descreve um modelo lógico proposto coerente com o planejamento do CentralBeer; ele não afirma que tabelas, migrations ou constraints já estejam criadas no banco.

10.2 Limitações do escopo

- Modelo dimensionado para uma loja e um caixa aberto por vez.
- Não há entidades de nota fiscal, SEFAZ, múltiplas filiais, venda fiada, entrega ou vasilhames retornáveis.
- Não há entidade de cliente porque o escopo atual não exige cadastro de consumidor para finalizar venda.
- Não há tabela de transação bancária/cartão, pois PIX, débito e crédito são processados externamente e apenas registrados no CentralBeer.
- Dados externos do Open Food Facts auxiliam o cadastro, mas a base local continua sendo a fonte utilizada durante a venda.

10.3 Decisões a confirmar na implementação

- Mapeamento definitivo de USUARIO: modelo padrão do Django com perfil complementar ou usuário customizado.
- Tamanhos finais de VARCHAR e nomenclatura física de tabelas/colunas geradas pelas migrations.
- Estratégia exata de constraints condicionais no SQLite quando uma regra precisar de validação adicional na camada de serviço.

10.4 Referência do projeto

Repositório consultado: <https://github.com/Bruno-Kowalski/Central-Beer>. Fonte funcional e técnica utilizada: README.md do CentralBeer, estado consultado em 03/10/2026.

10.5 Observação sobre os arquivos herdados

O próprio README identifica os PDFs existentes em docs/modelagem/banco-de-dados como materiais herdados do template que deverão ser substituídos por documentação específica do CentralBeer. Por essa razão, o conteúdo desses PDFs não foi usado na definição das entidades, atributos ou relacionamentos deste modelo.